Session S8
Génie Électrique
Codage de l'information

APP 3

Codage Entropique

Département de génie électrique et de génie informatique
Faculté de génie
Université de Sherbrooke

Automne 2026

Tous droits réservés © 2026 Département de génie électrique et de génie informatique, Université de Sherbrooke

Note : En vue d'alléger le texte, le masculin est utilisé pour désigner les femmes et les hommes.

Document S8_CI_APP3_2026_Guide_Etudiant.docx
Version Février 2007
Rédigé par Yakoub Ayeb
Modifié et Révisé par Hansa Devi Gukhool et Christian Royer
Révisé et modifié par Philippe Gournay (Automne 2026)
Tous droits réservés © 2026 Département de génie électrique et de génie informatique, Université de Sherbrooke.

Table des matières

1- Énoncé de la problématique..... 4

2- Notes de lecture 9

3- Références essentielles à consulter 13

4- Activités liées à la problématique 13

5- Productions attendues 14

6- Formation à la pratique procédurale 1 15

7- Formation à la pratique en laboratoire 17

8- Formation à la pratique procédurale 2 19

9- Validation pratique de la solution en laboratoire 21

10- Grille d'évaluation du rapport d'APP 22

1- Énoncé de la problématique

Mise en situation

Dans le cadre du contrat conclu avec HyperMédia, votre équipe poursuit le développement d'un outil de gestion multimédia permettant aux programmeurs d'accéder à distance à une base de données multimédia.

À la suite d'une première étape consacrée à la quantification des images, votre équipe doit maintenant concevoir un module permettant de stocker efficacement les images dans la base de données. Les images sont disponibles à quatre niveaux de qualité et de résolution, obtenus à l'aide d'un codec pyramidal.

HyperMédia souhaite regrouper ces quatre versions dans un même fichier afin de réduire les besoins de stockage et de faciliter leur gestion. Le système devra également permettre aux utilisateurs de récupérer la version d'une image correspondant au niveau de qualité et de résolution souhaité.

Votre équipe est chargée de concevoir et de réaliser une passerelle (voir Figures 1 et 2) entre le codec pyramidal et le système de stockage. Cette passerelle devra assurer deux fonctions principales :

- Compression et stockage : compresser indépendamment chacune des quatre sources à l'aide des algorithmes imposés, puis regrouper les quatre sources compressées dans un seul fichier dont la taille est aussi petite que possible.
- Récupération et décompression : permettre à un utilisateur de récupérer une source en précisant son nom et le niveau de qualité désiré, représenté par un entier de 1 à 4 (1 correspondant à la qualité et à la résolution minimales, et 4 aux maximales). La passerelle devra identifier la source demandée dans le fichier, reconnaître son format de compression et la décompresser à l'aide de l'algorithme approprié.

Contrairement à la quantification, qui peut réduire la précision de représentation des images, le codage entropique vise à réduire la taille des données sans perte d'information supplémentaire.

Pour des raisons liées aux droits et aux brevets, HyperMédia impose l'utilisation des trois algorithmes de codage suivants :

- Codage de Huffman ;
- Codage arithmétique ;
- Codage par plages (Run-Length Coding, ou RLC).

Les quatre sources d'images (source0.jpg à source3.jpg) vous seront fournies.

Vous devez développer un programme en MATLAB ou en Python qui réalise les opérations suivantes.

Partie 1 — Analyse des sources

1. Analyse statistique des sources

Pour chaque source, calculer les fréquences d'occurrence de chaque symbole ainsi que les fréquences d'occurrence conjointes de chaque paire de symboles. Représenter graphiquement les histogrammes correspondants et commenter les résultats.

2. Calcul des débits moyens minimum théoriques (entropie)

À partir des fréquences calculées, déterminer les débits moyens minimum théoriques (entropie H_i), pour un symbole et pour des paires de symboles, dans chaque source.

Partie 2 — Compression et stockage des images

3. Compression des sources

Compresser chaque source à l'aide des trois algorithmes imposés et calculer les débits moyens réels (R_i) obtenus après compression.

4. Comparaison des résultats

Comparer les entropies et les débits réels obtenus après compression. Pour chaque source et chaque algorithme, présenter l'entropie, le débit moyen après compression et le taux de compression (préciser votre définition de ce taux). Déterminer ensuite, pour chaque source, l'algorithme qui permet d'obtenir la plus petite taille de fichier compressé.

5. Regroupement des sources

Regrouper les quatre sources compressées dans un seul fichier. L'organisation du fichier doit permettre d'identifier chaque niveau de résolution par un entier de 1 à 4, ainsi que le format de compression utilisé pour la source correspondante.

6. Taux de compression global

Calculer le taux de compression global obtenu pour l'ensemble des quatre niveaux de résolution.

Les points 1 à 6 doivent être documentés dans le rapport d'APP. Consultez la grille d'évaluation du rapport pour connaître les éléments attendus.

Partie 3 — Décompression et récupération des images

Une fois les sources compressées et regroupées dans un fichier, celui-ci est stocké sur une unité de disque. Vous devez compléter votre programme en réalisant la fonction de décompression.

Le programme de décompression doit effectuer les opérations suivantes.

7. Paramètres d'entrée

Recevoir le nom de la source et un entier compris entre 1 et 4 indiquant le niveau de résolution désiré.

8. Lecture du fichier

Lire dans le fichier les données correspondant au niveau de résolution demandé.

9. Identification du format

Reconnaître le format de compression utilisé pour les données sélectionnées.

10. Décompression

Décompresser les données à l'aide de l'algorithme approprié.

11. Affichage et comparaison

Afficher l'image obtenue et la comparer à la source originale correspondante.

Précision concernant le codage pyramidal : normalement, la reconstruction d'une image à un niveau de résolution supérieur à 1 nécessite également les sources correspondant aux niveaux inférieurs. Toutefois, le décodeur pyramidal ne vous est pas fourni. Vous n'avez donc pas à reconstruire l'image à partir des différents niveaux : il suffit de décompresser et d'afficher la source correspondant au niveau demandé.

Partie 4 — Étude de faisabilité du codage de fichiers texte

HyperMédia envisage d'ajouter un service permettant d'associer à chaque source d'image un fichier texte contenant son historique. L'entreprise souhaite utiliser l'algorithme LZ77 pour compresser ces données.

Afin d'évaluer la faisabilité de ce projet, vous devez réaliser un prototype de programme mettant en œuvre l'algorithme LZ77 et le tester sur un fichier texte de votre choix.

Seule la fonction de compression du fichier texte est exigée.

Partie 5 — Questions complémentaires

En complément du programme, HyperMédia souhaite obtenir des réponses aux questions suivantes. Les réponses doivent être intégrées au rapport d'APP.

1. Modèles de probabilités et algorithmes de codage

Quels modèles de probabilités sont les plus appropriés pour le codage de Huffman, le codage arithmétique et le codage RLC ? Répondez en une page, en justifiant vos explications par un exemple pour chacune des trois approches.

2. Comparaison de LZ77 et du codage de Golomb

Décrivez, en une page, les différences entre les algorithmes LZ77 et Golomb, leurs avantages et leurs inconvénients, ainsi que leurs principaux cas d'utilisation.

3. Codage de Fibonacci

Présentez les principes du codage de Fibonacci et comparez cette méthode au codage de Golomb.

4. Analyse et amélioration du taux de compression

Commentez le taux de compression global obtenu et proposez des pistes d'amélioration pour vos algorithmes de compression. En une page, décrivez les différentes étapes que vous entreprendriez pour améliorer les résultats et justifiez chacune de vos propositions.

Les réponses à ces quatre questions font partie des éléments à fournir dans le rapport d'APP. Consultez la grille d'évaluation du rapport pour plus de détails.

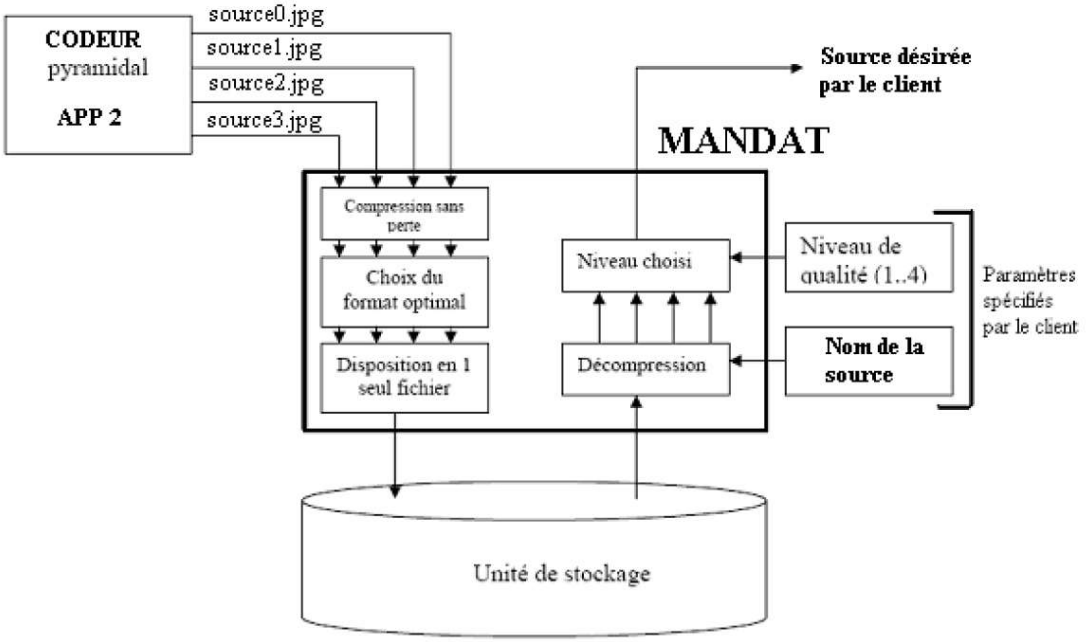


Figure 1: Schéma bloc de la passerelle

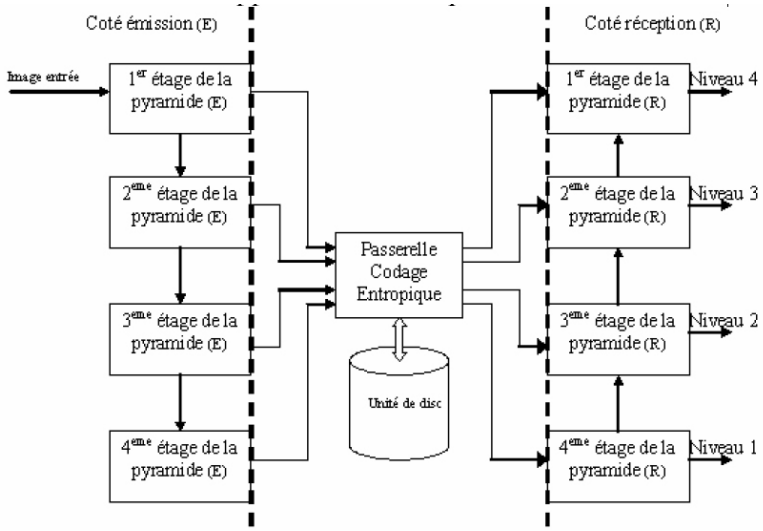


Figure 2: Disposition de la passerelle par rapport au CODEC pyramidal

2- Notes de lecture

2.1 Éléments théoriques

Les notes suivantes sont un concentré des éléments théoriques principaux nécessaires à la réalisation de la problématique. Les étudiants sont priés de se référer au chapitre 2 et à l'annexe A du manuel référence (Khalid Sayood) pour approfondir l'aspect mathématique (raisonnements et démonstrations mathématiques).

1. Le codage est l'opération d'assigner des séquences binaires aux éléments d'un alphabet. L'ensemble des séquences binaires est appelé *code,* et les membres du code sont des *mots de code.* Un *alphabet* est l'ensemble des symboles produits par la source.

2. La quantité d'informations H_i véhiculée par un message dépend de sa probabilité d'apparition. Cette quantité est donnée par :
$$H_i = \log_2\left(\frac{1}{p_i}\right) = -\log_2(p_i)$$

3. L'espérance mathématique des quantités d'information donne l'entropie :
$$H = E(H_i) = -\sum_i p_i \log_2(p_i)$$

4. L'entropie est maximale lorsque tous les symboles sont équiprobables, on a alors :
$$p_i = \frac{1}{n} \Rightarrow H = -\log_2(p_i)$$

5. Pour toute source discrète il existe un code uniquement décodable vérifiant la relation :
$$H \leq l_{moy} \leq H + 1$$

l_{moy} est la longueur moyenne du code.

6. En regroupant n symboles d'une même source, la relation entre la longueur moyenne et l'entropie devient :
$$H \leq \frac{l_{moy}}{n} \leq H + \frac{1}{n}$$

7. Inégalité de McMillan-Kraft :
$$\sum_{i=1}^{n} 2^{-l_i} \leq 1 \text{ ; } l_i \text{ : longueur du code i.}$$
Cette inégalité signifie qu'un code est unique si et seulement si la relation de Kraft-Mcmillan est vérifiée.

8. Mesure de la distorsion :
L'erreur quadratique moyenne de deux images I_1 et I_2 est donnée par :
$$e = \sum_i \sum_j \left(I_1(i,j) - I_2(i,j)\right)^2$$
Le PSNR (Peak SNR) est donnée par :
$$PSNR = 10\log_{10}\left(\frac{PixelMaximal * PixelMaximal}{e}\right)$$

2.2 Résumés des approches des codages entropiques

Codage HUFFMAN

Le code HUFFMAN est un code optimal à préfixe. Ce code est basé sur deux principes :

- Les symboles très fréquents se voient attribués des codes courts et les symboles moins fréquents des codes longs.
- Les deux symboles les moins fréquents ont des codes de mêmes longueurs.

Le code HUFFMAN se construit en deux passes. Une première passe pour calculer les probabilités d'occurrence de chaque symbole et une deuxième passe pour attribuer un code à chaque symbole.

L'algorithme de codage consiste à construire un arbre binaire en fonction des probabilités d'occurrences. Le code de chaque symbole est alors obtenu en parcourant l'arbre. Pour chaque symbole, le passage par chaque branche est unique et engendre l'ajout d'un «1 » ou d'un «0 »au code.

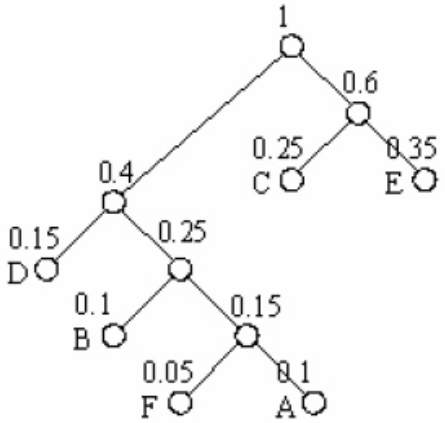
Il existe plusieurs variantes du code HUFFMAN. La plus importante est la version adaptative de l'algorithme de HUFFMAN qui génère les codes en une seule passe.

Exemple :

Soit à coder l'alphabet {A, B, C, D, E, F} dont les symboles ont les probabilités suivantes :

p(A)=0.1, p(B)=0.1, p(C)=0.25, p(D)=0.15, p(E)=0.35, p(F)=0.05.

Symboles	Probabilités	Codes
A	0.1	0111
B	0.1	010
C	0.25	10
D	0.15	00
E	0.35	11
F	0.05	0110



Codage arithmétique

Le codage arithmétique est plus performant que le codage HUFFMAN pour les sources à alphabet réduit et de distribution de fréquences d'occurrences déséquilibrées. Le codage arithmétique représente le message à coder par un nombre réel unique dans l'intervalle [0,1]. À mesure que le message s'allonge, l'intervalle de représentation se réduit et le nombre de bits s'accroît.

Exemple :

Soit à coder le message « BILL GATES »
Le modèle de probabilité de ce message est :
p(L) = 0.2, p(SPACE)= p(A) =p(B)= p(E)= p(G)= p(I)= p(S)= p(T)=0.1.
On commence par assigner un intervalle entre 0 et 1 pour chaque symbole :

SPACE	0.1	0.00 <= r < 0.10
A	0.1	0.10 <= r < 0.20
B	0.1	0.20 <= r < 0.30
E	0.1	0.30 <= r < 0.40
G	0.1	0.40 <= r < 0.50
I	0.1	0.50 <= r < 0.60
L	0.2	0.60 <= r < 0.80
S	0.1	0.80 <= r < 0.90
T	0.1	0.90 <= r < 1.00

On désigne par L la limite inférieure de l'intervalle de codage et H sa limite supérieure :

Symbole	L	H
B	.2	.3
I	.25	.26
L	.256	.258
L	.2572	.2576
<space>	.25720	.25724
G	.257216	.257220
A	.2572164	.25722168
T	.25721676	.25722168
E	.257216772	.2572216776
S	.2572167752	.25722167756

Codage LZ (Lempel-Ziv)

Le codage LZ a constitué une révolution dans le domaine de la compression et du codage. En effet, pour la première fois après le code de HUFFMAN, les méthodes statistiques ont été remises en question. Le codage LZ part d'une idée très simple. Le dictionnaire est construit au fur et à mesure que le message est codé. Chaque séquence est comparée aux séquences présentes dans le dictionnaire. Si elle s'y trouve déjà, on se contente de transmettre sa position dans le dictionnaire. Sinon elle est rajoutée au dictionnaire.

Le codage LZ est à la base de plusieurs programmes d'archivage tels que : COMPRESS,

PKZIP et ARJ.

Exemple :
Soit à coder le message « abracadabra »
La séquence est codée par 101100110111101100000
Ce résultat est obtenu en commençant par un dictionnaire contenant les caractères (a, b, c, d, r) et faisant glisser une fenêtre à travers tout le message.

Codage RLC (Run Length Coding)

RLC est un algorithme de codage simple et rapide. Néanmoins, cette méthode est performante pour les sources dont les symboles ont des probabilités d'apparition très proches.

RLC réduit la taille physique d'une répétition de symboles. La répétition de symboles est généralement codée avec deux octets. Le premier octet représente le symbole et le deuxième octet représente le nombre de répétitions.

Exemple :
Soit à coder la séquence : AAAAAABBBCCCCCC
Le résultat du codage par RLC est : A6B3C6.

Codage GOLOMB

Ce code est fait pour les nombres entiers positifs. Le code dépend du choix d'un paramètre m. Pour coder l'entier *n*, la première étape est de calculer les quantités suivantes :

$$q = \left\lfloor \frac{n}{m} \right\rfloor, \quad r = n - qm \quad et \quad c = \lceil log_2(m) \rceil$$

Le code est formé de 2 parties. La première partie est le code unaire (une suite de symboles « 1 » suivi d'un seul symbole « 0 » utilisé comme délimiteur) de q (code à longueur variable).
La seconde partie est le code binaire à c bits (donc de longueur fixe) de r.

Exemple :
m=3;

n	0	1	2	3	4	5	6	7
code	000	001	010	1000	1001	1010	11000	11001

3- Références essentielles à consulter

Manuel de référence:

« Introduction to data compression » de Khalid Sayood, Quatrième ou Cinquième édition

Procédural 1	Chapitre 2 : • Sections 2.1 à 2.4.3	24 pages (*)
	Chapitre 3 : • Sections 3.1 à 3.2.4 • Sections 3.5 et 3.6	14 pages (*) 3 pages (*)
	Chapitre 4 : • Sections 4.1 à 4.3	12 pages (*)
Procédural 2	Chapitre 4 : • Sections 4.4	17 pages (*)
	Chapitre 5 : • Sections 5.1 à 5.4	16 pages (*)
	Total :	**86 pages** (*)

(*) Nombres de pages pour la quatrième édition. Peut différer légèrement pour la cinquième.

Remarque importante :

On vous conseille d'essayer de résoudre les exemples d'exercices dans vos lectures afin de vous familiariser davantage avec la matière.

4- Activités liées à la problématique

Semaine 1 :

- Tutorat d'ouverture
- Étude personnelle et exercices
- Formation à la pratique procédurale 1
- Formation à la pratique en laboratoire

} *Vous devez vous préparer individuellement avant de vous présenter à ces trois activités.*

Semaine 2 :

- Étude personnelle et exercices
- Formation à la pratique procédurale 2
- Consultation pour la résolution de la problématique

Semaine 3 :

- Validation pratique (non-évaluée) de la solution (équipes de 2)
- Remise du rapport d'APP et tutorat de fermeture
- Étude personnelle et exercices
- Évaluation sommative

5- Productions attendues

1- Rapport d'APP à remettre le lundi 16 novembre 2026 (veille du Tutorat 2) avant minuit. Ce rapport doit contenir les éléments décrits dans la grille d'évaluation à la fin de ce document. La longueur recommandée (excluant la page de garde, la table des matières et les annexes) est de 12 pages.

2- **À faire avant l'activité de validation de la solution :**

- Avoir bien réfléchi à tous les éléments de solution de la problématique.
- Avoir une version modulaire (une fonction par module) et bien documentée du code Matlab ou Python servant à résoudre la problématique.
- Avoir sauvegardé au préalable les fichiers sources de démonstration pour illustrer vos résultats pour les différentes méthodes étudiées.

6- Formation à la pratique procédurale 1

Buts de l'activité :

- Maîtriser le calcul du modèle de probabilité d'une source et l'interprétation de ce modèle.
- Calcul de l'entropie du 1er ordre et du 2ème ordre.
- Calcul du débit moyen d'un code.
- Calcul de l'efficacité d'un code.
- Maîtriser le codage Huffman, RLC et arithmétique.

Problème 1

Soit X une variable aléatoire. Les valeurs de X se trouvent dans un alphabet de M lettres. Démontrez que l'entropie de X se trouve à l'intérieur des bornes suivantes :

$$0 \leq H(X) \leq \log_2 M$$

Problème 2

Une source possède un modèle de probabilité donné par :

$P = \{p_0, p_1, p_2, p_3, \ldots p_m\}$ avec une entropie H_p.

Une autre source possède un modèle de probabilité donné par :

$Q = \{q_0, q_1, q_2, q_3, \ldots q_m\}$ avec une entropie H_q où :

$q_i = p_i$ pour i = 0,1, 2…,j-2,j+1,…m.

$q_{j-1} = q_j = 0.5\ (p_j + p_{j-1})$.

Démontrer que H_q est supérieure ou égale à H_p.

Problème 3

Soit X une variable aléatoire qui prend ses valeurs dans l'alphabet A qui contient 4 lettres : $A = \{a_1, a_2, a_3, a_4\}$.

a) Calculez l'entropie d'ordre 1 de X dans les cas suivants :
   1. $p(a_1) = p(a_2) = p(a_3) = p(a_4) = 0.25$.
   2. $p(a_1) = 0.5$; $p(a_2) = 0.25$; $p(a_3) = p(a_4) = 0.125$.
   3. $p(a_1) = 0.505$; $p(a_2) = 0.25$; $p(a_3) = 0.125$; $p(a_4) = 0.12$.
b) Calculez l'entropie d'ordre 2 de X dans les deux cas suivants :
   1. $p(a_1) = p(a_2) = p(a_3) = p(a_4) = 0.25$.
   2. $p(a_1) = 0.5$; $p(a_2) = 0.25$; $p(a_3) = p(a_4) = 0.125$.
c) Commentez sur la valeur des deux entropies obtenues

Problème 4

Dannick est cofondateur et directeur scientifique de la société GénieSpace. Son robot d'exploration martienne est conçu pour répondre à quatre commandes : avancer, reculer, tourner à droite, prélever un échantillon. Son robot devant répondre immédiatement lors de la réception de ces commandes (sans avoir à attendre la commande suivante), il doit élaborer un code qui soit à la fois uniquement décodable et décodable de façon instantanée. Son stagiaire Félix-Antoine lui propose cinq codes potentiels :

1. {0, 01, 11, 111}
2. {0, 01, 110, 111}
3. {0, 01, 011, 111}
4. {0, 10, 110, 111}
5. {1, 10, 110, 111}

a) Aidez Dannick à déterminer lesquels de ces codes sont décodables de manière unique.
b) Pour les codes qui ne sont pas décodable de manière unique, déterminez s'il serait possible de les modifier pour les rendre uniquement décodables.
c) Pour ceux qui sont déjà décodables de manière unique, identifiez ceux qui sont instantanés.

Problème 5

Le robot martien de Dannick vient de découvrir des traces de vie extraterrestre. On sait que l'ADN humain est une longue chaine dans laquelle alternent quatre bases azotées : la cytosine (C), la guanine (G), la thymine (T) et l'adénine (A). Surprise ! l'ADN martien comporte une cinquième base appelée l'uracile (U). Dannick appelle son amie Catherine, cofondatrice et directrice scientifique de la compagnie GénieTique, et lui confie le mandat de compresser sans perte l'information recueillie par le robot.

Les probabilités d'occurrence des bases azotées sont :

$$p(C) = 0.2; \quad p(G) = 0.1; \quad p(T) = 0.2; \quad p(A) = 0.1; \quad p(U) = 0.4$$

a) Aidez Catherine en calculant l'entropie d'ordre 1 de cette source.
b) Construisez l'arbre de Huffman pour cette distribution de probabilité en utilisant :
   1. La méthode conventionnelle;
   2. La méthode pour minimiser la variance de la longueur des mots de codes.
c) Construisez la table de codes de Huffman correspondant à chacun des 2 arbres Huffman.
d) Calculez la longueur moyenne et la redondance des codes générés avec les deux méthodes. Comparez ces résultats.
e) Encodez la séquence [U T C T U U C U] avec chacun des deux codes. Comparez le débit moyen par symbole réel pour cette séquence particulière avec l'entropie d'ordre 1 calculée en a). Comment est-ce possible ?

Problème 6

Joséanne, une scientifique au laboratoire de recherche de GénieTique, a entendu parler du codage arithmétique. Elle élabore le modèle de probabilité suivant pour les séquences d'ADN extraterrestre :

$$p(U) = 0.2; \; p(T) = 0.3; \; p(C) = 0.5$$

a) En utilisant l'algorithme arithmétique, calculez le « tag » correspondant à la séquence U U C T C U en utilisant :
   1. La méthode conventionnelle;
   2. La méthode avec du « scaling ».
b) Écrivez le « tag » sous forme binaire.

7- Formation à la pratique en laboratoire

But de l'activité :

- Pouvoir concevoir et mettre en œuvre un code pour la compression entropique.

Problème 1

Voici une portion de texte :

AAAAAAABBBBCCCDDDDDDDDDDEEEEEAAAAADDDDCCCCCC
CCEEEEEEEEEEEAAAAAABBBBEEDDDDAAAAACCCCCCCCCCC

Les caractères de ce texte peuvent être représenté par une variable aléatoire Y qui prend ses valeurs dans un alphabet W : W = {A, B, C, D, E}. Monsieur X vous demande de coder ce fichier avec le codage RLC. Les 5 premiers bits d'un octet représenteront le caractère et les 3 derniers bits représenteront le nombre de répétition du caractère. Après avoir codé le texte vous devez l'enregistrer dans un fichier texte. Vous devez bien spécifier les paramètres nécessaires pour extraire le texte codé lors du décodage.

Réalisez un code Matlab ou Python pour :

a) Coder la portion de texte avec l'algorithme RLC
b) Enregistrer le texte codé dans un fichier.
c) Décoder le texte codé enregistré dans le fichier. Est-ce que la séquence décodée est similaire à la séquence originale?

Problème 2

Le modèle de probabilité d'une variable aléatoire X est :

$p(U) = 0.2; \quad p(T) = 0.3; \quad p(C) = 0.5$

La séquence suivante sera codée en utilisant l'algorithme arithmétique conventionnel :

U U C T C U

Une fois la séquence codée vous devrez l'enregistrer dans un fichier.

Réalisez un code Matlab ou Python pour :

a) Calculer la fonction densité cumulative (F_x)
b) Calculer le « tag » correspondant à la séquence.
c) Écrire le « tag » sous forme binaire afin de permettre son enregistrement dans le fichier. Est-ce que vous obtenez la même valeur que le problème 6 du procédural ?

Problème 3

Soit X une variable aléatoire qui prend ses valeurs dans un alphabet Z : Z = {a, b, c, d, e, f, g}. Ces lettres de l'alphabet sont recueillies d'un certain fichier texte. Le tableau suivant résume l'occurrence de ces lettres dans le fichier de texte.

Lettre	Occurrence
a	10
b	10
c	20
d	30
e	50
f	50
g	80

Réalisez un code Matlab ou Python pour :

a) Calculer les probabilités d'apparitions des symboles.
b) Représenter graphiquement ces probabilités à l'aide des histogrammes.
c) Donner l'entropie de la source (1er ordre et 2ème ordre)
d) Générer l'arbre de Huffman pour le 1er ordre.

8- Formation à la pratique procédurale 2

Buts de l'activité :

- Initiation au décodage par l'algorithme arithmétique et au codage par l'algorithme LZ.

Problème 7

Les symboles d'un alphabet possèdent les occurrences suivantes :

Symbole	Occurrence
A	37
B	38
C	25

Décodez le « tag » suivant : 0.63215699.

Problème 8

Vincent et Mathys, tous deux étudiants de dernière année à l'Université de Sherbrooke, décident de mettre à l'épreuve le croupier du casino de leur voisinage. Ils décident d'utiliser le code de Golomb avec m = 5 pour communiquer le total de leurs cartes en main.

Si Vincent a 10 et Mathys a 21, quels sont les mots de code qu'ils doivent transmettre ?

Problème 9

Considérons l'exemple présenté dans le chapitre 5.2 du livre de Sayood. La probabilité qu'un mot particulier de 4 lettres appartienne au dictionnaire est p. Ainsi, le nombre moyen de bits par mot R est donné par :

$$R = 9p + 21(1-p) \ldots \text{Équation (5.1)}$$

a) Modifiez l'équation pour obtenir le débit R en fonction de la taille du dictionnaire M et de la probabilité p.
b) En assumant que tous les mots sont équiprobables et indépendants, dessinez la relation entre la taille M du dictionnaire et le débit R.

Problème 10

Une approche de codage par dictionnaire courante est celle du digramme (Sayood 5.3.1). Une source prend ses valeurs dans l'alphabet A qui consiste de 5 lettres : A = {r, s, t, u, a}. La séquence suivante doit être encodée par digramme: rsartrursar.

a) Construisez le dictionnaire (aucune limite quant à la taille du dictionnaire n'est imposée).
b) Codez la séquence donnée.

Problème 11

Newton, espion du MI6 au service de Sa Majesté, doit communiquer à Pape Malick les coordonnées d'un contact qui doit leur transmettre les plans secrets d'un quantificateur révolutionnaire développé par le professeur Gournay. Ils ont convenu d'utiliser l'algorithme LZ77 pour communiquer. Ils ont également convenu d'utiliser l'alphabet suivant : A, B, C, N, P.

Aidez Newton à encoder la séquence suivante :

ANNABANANAACAPACABANAAAAAA

Problème 12

Touan a codé pour William un message très important. Il a utilisé l'algorithme LZW, avec comme dictionnaire initial :

Index	Caractère
1	a
2	<esp>
3	r
4	t

La sortie du codeur était :

3 1 4 6 8 4 2 1 2 5 10 6 11 13 6

a) Décodez cette séquence.
b) Encodez de nouveau le message obtenu et décodez-le en utilisant le même dictionnaire initial. Obtenez-vous le même message ?

9- Validation pratique de la solution en laboratoire

Buts de l'activité :
La validation (non évaluée) se fait en groupe de 2 :

- Validez en laboratoire la solution de la problématique :
    1. L'approche Huffman
    2. L'approche Arithmétique
    3. L'approche RLC
    4. L'approche LZ77
    5. L'implémentation de tout le système compression/décompression
- Pour chaque code, démontrez l'effet du codage par la comparaison des tailles d'une source avant et après le codage.
- Démontrez par un exemple le fonctionnement de la passerelle selon le cahier de charges demandé.

10- Grille d'évaluation du rapport d'APP

Éléments du rapport	GEI–780 #2	GEI–780 #3	GEI–780 #4
Qualité de la présentation (organisation du rapport, introduction et conclusion, syntaxe, grammaire et orthographe)	20		
Analyse statistique (distributions et entropies d'ordre 1 et 2, discussion et commentaires)	20		
Code RLC (schéma-bloc, explications, performance obtenue + qualité de l'implémentation)	20	5	
Code de Huffman (schéma-bloc, explications, performance obtenue + qualité de l'implémentation)	20	5	
Code arithmétique (schéma-bloc, explications, performance obtenue + qualité de l'implémentation)	20	5	
Algorithme LZ77 (schéma-bloc, explications, performance obtenue sur le texte à compresser)	10		
Réponses aux questions complémentaires (fournir toutes les explications, les schémas et les équations nécessaires)			10
Code source modulaire et bien commenté (sous forme d'une archive zip jointe)		5	
TOTAL	90 points	20 points	10 points

Le rapport doit être rédigé de façon professionnelle. Sa présentation doit être soignée et conforme aux règles du département. Il doit contenir une introduction et une conclusion. Les explications doivent être claires et complètes. Les figures et les tables doivent être numérotées. Enfin, la syntaxe, la grammaire et l'orthographe doivent être respectées. Une pénalité allant jusqu'à 20 points pourra être imposé pour manquement à ces règles.